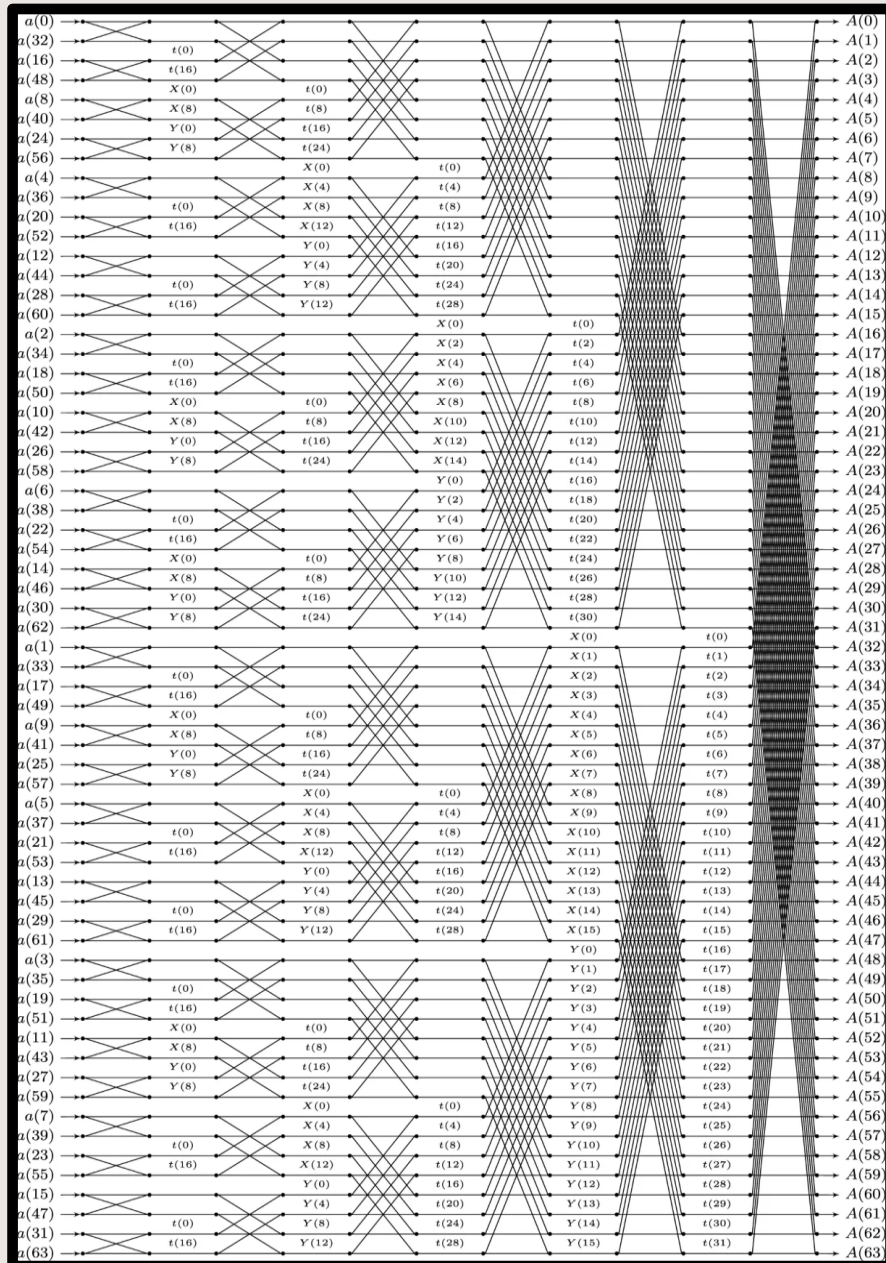


TP_MP25

16조

한민준, 신준우

64-Point FFT Butterfly Flow Diagram



RISC-V Register Setup

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–7	t0–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller
f0–7	ft0–7	FP temporaries	Caller
f8–9	fs0–1	FP saved registers	Callee
f10–11	fa0–1	FP arguments/return values	Caller
f12–17	fa2–7	FP arguments	Caller
f18–27	fs2–11	FP saved registers	Callee
f28–31	ft8–11	FP temporaries	Caller

```

// FFT (DIT)
for (set = 0; set < N_SET; set++)
{
    // load_input
    for (j = 0; j < L_FFT; j++)
    {
        int rev = 0;
        for (b = 0; b < N_STAGE; b++)
        {
            rev |= ((j >> b) & 1) << (N_STAGE - 1 - b);
        }
        SRAM_0[rev] = input_region_of_SRAM_IO[set * L_FFT + j];
    }
    //
    // compute_fft
    n = 1 << N_STAGE;
    l = 1;
    k = N_STAGE - 1;
    while (l < n)
    {
        istep = l << 1;
        for (m = 0; m < l; ++m)
        {
            j = m << k;
            twiddle = SRAM_W[j];
            for (i = m; i < n; i += istep)
            {
                j = i + l;
                dpath_reg_0 = SRAM_0[i];
                dpath_reg_1 = SRAM_0[j];
                dpath_reg_2 = twiddle;

                R2_FFT_Engine();

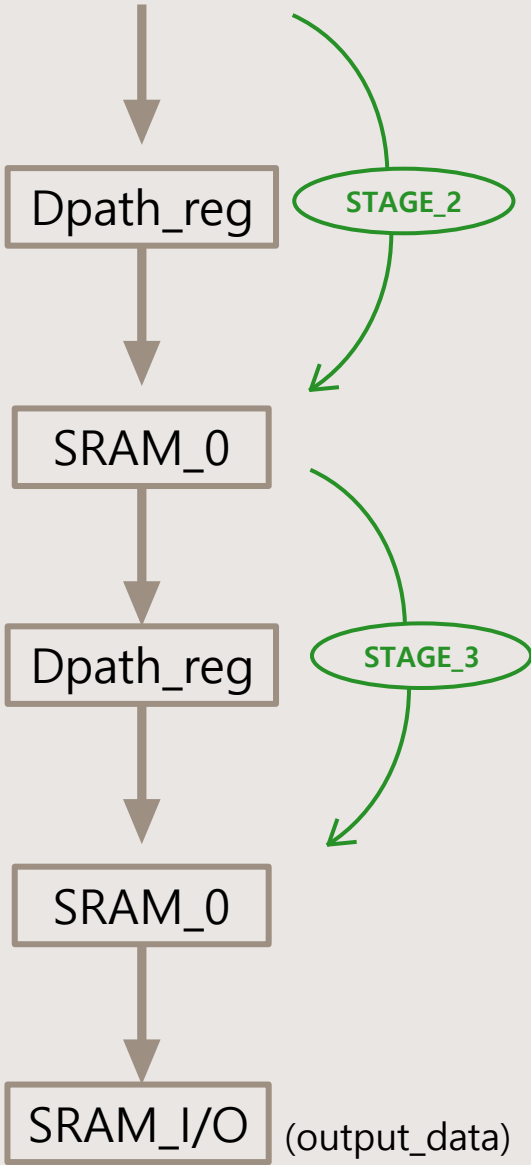
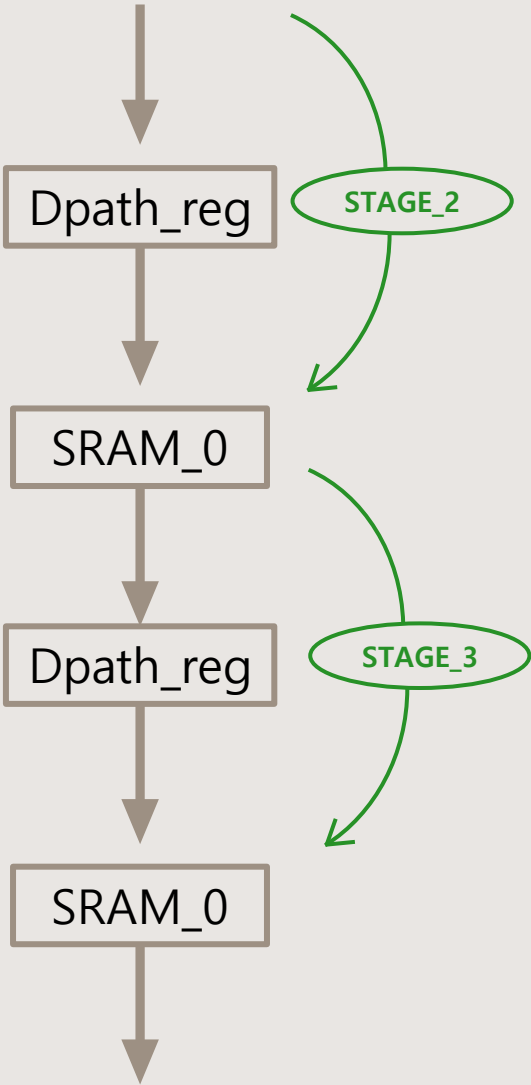
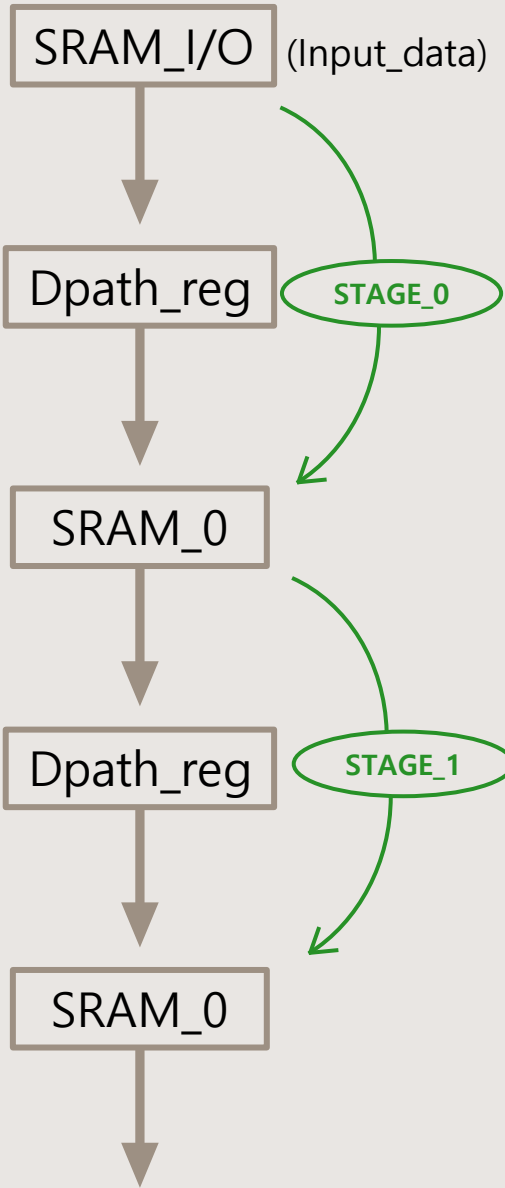
                SRAM_0[i] = dpath_reg_0;
                SRAM_0[j] = dpath_reg_1;
            }
        }
        //
        #ifdef DEBUG
        // example codes for debug
        if (set == 0 && l == 1)
        {
            printf("State of SRAM_0 after first stage of first set\n");
            for (j = 0; j < L_FFT; j++)
                printf("SRAM_0[%2d] = %t0x%8x\n", j, SRAM_0[j]);
        }
        //
        #endif

        --k;
        l = istep;
    }
    //
    // store_output
    for (j = 0; j < L_FFT; j++)
        output_region_of_SRAM_IO[set * L_FFT + j] = SRAM_0[j];
    //
}

```

코드의 latency를 줄이기 위한 전략

1. STAGE_0에 한해서만 SRAM_0를 거치지 않고 SRAM_I/O → Dpath_reg로 바로 lw, sw 함으로써, 코드의 latency를 감소시킴
2. SRAM_I/O, SRAM_0, SRAM_W의 주소계산을 미리 계산해둔 Table에 따라 offset을 넣어놓음으로써, 주소계산을 위한 loop, branch, Jump inst을 완전히 삭제
3. 한 stage내에서 같은 twiddle factor들끼리 연산을 한번에 진행함으로써 Twiddle factor load, store를 위한 메모리 접근 횟수를 최소화



Stage_0

SRAM_I/O → Dpath_reg → SRAM_0

```

# Butterfly 30: SRAM_I0[15] and SRAM_I0[47] --> SRAM_0[60], SRAM_0[61]
lw      t2, 60(s0)          # t2 = SRAM_I0[15]
lw      t3, 188(s0)         # t3 = SRAM_I0[47]
sw      t2, 0(s4)           # dpath_reg_0 = SRAM_I0[15]
sw      t3, 0(s5)           # dpath_reg_1 = SRAM_I0[47]
sw      t4, 0(s6)           # dpath_reg_2 = W[0]
lw      t2, 4(s4)           # t2 = dpath_reg_0 (X = A + WB)
lw      t3, 4(s5)           # t3 = dpath_reg_1 (Y = A - WB)
sw      t2, 240(s2)         # SRAM_0[60] = X
sw      t3, 244(s2)         # SRAM_0[61] = Y

```

SRAM_I/O, SRAM_0, SRAM_W의 주소계산을 미리 계산해둔 Table에 따라
offset을 넣어 놓음으로써, 주소계산을 위한 branch, jump를 완전히 삭제

→ Loop unrolling

```
#-----  
# Stage_1: SRAM_0 <--> FFT_engine (l=2, m=0~1)  
#-----
```

twiddle factor W[0],W[16] 그룹화로 SRAM_W 접근 2회로 최소화

```
#-----  
# Stage_2: SRAM_0 <--> FFT_engine (l=4, m=0~3)  
#-----
```

twiddle factor W[0],W[8],W[16],W[24] 사용, 오프셋 하드코딩으로 효율화

```
#-----  
# Stage_3: SRAM_0 <--> FFT_engine (l=8, m=0~7)  
#-----
```

twiddle factor 그룹화 및 오프셋 하드코딩으로 메모리 접근 최적화

```
#-----  
# Stage_4: SRAM_0 <--> FFT_engine (l=16, m=0~15)  
#-----
```

동일한 방법으로 twiddle factor load 16회로 최소화

```
#-----  
# Stage_5: SRAM_0 <--> FFT_engine (l=32, m=0~31)  
#-----
```

twiddle factor W[0] to W[31] 사용, 그룹화로 SRAM_W 접근 최적화

한 stage내에서 같은 twiddle factor들끼리 연산을 한번에 진행함으로써
Twiddle factor load, store를 위한 메모리 접근 횟수를 최소화